UNIVERSITY OF ENGINEERING AND TECHNOLOGY (UET), LAHORE

Department of Computer Science

Semester Project

Semester 1 — Course: Programming Fundamentals (CS-101)

Project Title:

PAW-SOME PETS & CO.

A Digital Management & Adoption Ecosystem

Submitted By: Muhammad Uzair Shakeel **Roll No:** 2026(S)-CS-96

BS Computer Science (Section B)

Submitted To: Professor Laeeq Khan Niazi

Date:

INTRODUCTION & PROJECT SCOPE

Project Introduction

"Paw-Some Pets & Co." is a custom-built Command Line Interface (CLI) application developed to solve the logistical challenges of modern animal shelters. The system bridges the gap between shelter administrators, community donors, and potential pet adopters by centralizing data management into a single, cohesive platform.

Problem Statement

Manual record-keeping in animal shelters often leads to data fragmentation, loss of financial transparency, and difficulty in matching pets with the right owners. This project automates these processes to ensure every pet's health, history, and adoption status are tracked with 100% accuracy.

3. Project Objectives

- To implement a secure, multi-role access system.
- To utilize efficient data structures (Parallel Arrays) for high-speed data retrieval.
- To provide real-time financial auditing through a Transaction History log.
- To ensure a user-friendly experience through a consistent and aesthetic CLI design.

1. Functional Requirements

1.1 System Overview

The "Paw-Some Pets & Co." application is a comprehensive Pet Management and Adoption System designed to streamline shelter operations. It supports three distinct user roles: Administrative Staff, Pet Donors, and Customers/Adopters.

1.2 Administrative Requirements

The Admin module serves as the core management hub, requiring a secure password for access.

- **Inventory Management:** Admin can view the full list of pets and update specific details (Name, Age, Health, etc.) for any record.
- **Data Organization:** Ability to sort pets by age in both ascending and descending order using the Bubble Sort algorithm.
- **Record Maintenance:** Admin can manually delete records or manage shelter capacity.
- **Financial Auditing:** Access to separate "Donation Bank" and "Adoption Revenue" trackers to monitor shelter funds.
- **Business History:** A transaction log that retains the last 10 financial events for auditing purposes.

1.3 Pet Donor Requirements

The Donor module allows members of the public to contribute to the shelter's mission.

- **Pet Intake:** Users can register a new pet into the system by providing Name, Type, Breed, Age, Health status, and personality traits.
- **Financial Support:** Donors can contribute any amount of money to the general shelter fund.
- **Capacity Awareness:** Donors can check if the shelter has space available before attempting to donate a pet.

1.4 Customer / Adopter Requirements

The Customer module focuses on matching pets with new homes.

- **Pet Discovery:** Users can browse the entire inventory or search for a specific pet by name using a case-insensitive search engine.
- **Adoption Process:** Customers can select a pet for adoption, which triggers a fixed adoption fee (1000 PKR).
- **Automated Inventory Update:** Upon successful adoption, the system automatically removes the pet from the active inventory and logs the transaction.

2. Data Structures

2.1 Parallel Arrays

Since the project is developed in C++ without an external database, the system utilizes Parallel Arrays to manage pet records. Each attribute of a pet is stored in a separate array, linked by a common index:

- **`string petNames[1000]:`** Stores the unique identifiers for each animal.
- **`string petTypes[1000] & petBreeds[1000]:`** Categorical data for filtering.
- **`int petAges[1000]:`** Numerical data used by the Sorting Algorithm.
- **`float petHealth[1000]:`** Precision data representing the physical condition of the pet.

2.2 Transaction History Buffers

To maintain a secure audit trail, the system implements a Rolling Buffer structure:

- **`string historyType[10]:`** Categorizes transactions as "Adoption" or "Donation".
- **`double historyAmount[10]:`** Tracks the financial value of each event.
- **`int historyCount:`** A global pointer that ensures only the most recent 10 transactions are displayed, optimizing memory usage.

2.3 Algorithm — Bubble Sort

The system uses the Bubble Sort algorithm for organizational tasks. By comparing adjacent elements in the petAges array and swapping them alongside their corresponding data in the parallel arrays, the system ensures data integrity is maintained during sorting operations.

3. Wireframes (Interface Design)

Below are the primary interfaces of the Paw-Some Pets & Co. management system, demonstrating the 52-character symmetrical CLI design and standardized data presentation.

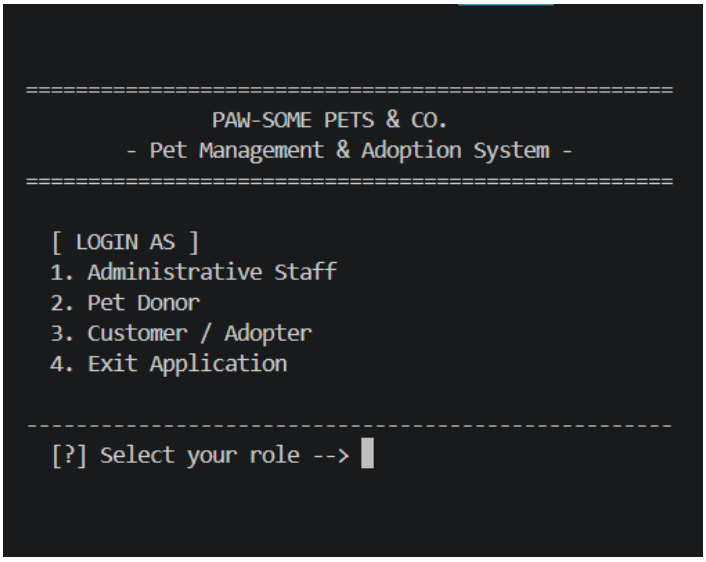

Figure 1: *Role-selection gateway with unified branding header.*

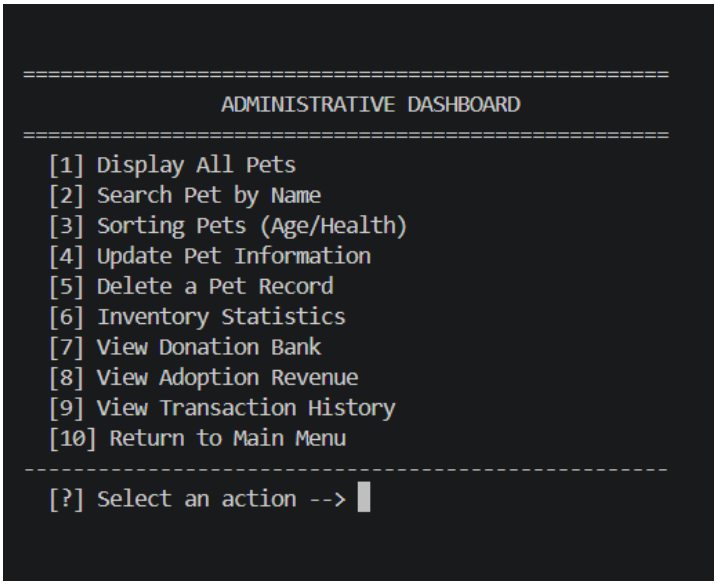

Figure 2: *Administrative control panel with bracketed navigation.*

```
==============================================
           CURRENT PET INVENTORY
==============================================

Total Pets: 4
[ ID: 1 ]---------------------------------------
  Name   : Whiskers
  Type   : Domestic (Persian)
  Status : 2 yrs | Health: 92.5%
  Traits : Calm
[ ID: 2 ]---------------------------------------
  Name   : Tracker
  Type   : Field & Sport (Pointer)
  Status : 3 yrs | Health: 98%
  Traits : Fast Tracker
[ ID: 3 ]---------------------------------------
  Name   : Duke
  Type   : Field & Sport (Labrador)
  Status : 2 yrs | Health: 99%
  Traits : Great Swimmer
[ ID: 4 ]---------------------------------------
  Name   : Luna
  Type   : Domestic (Siamese)
  Status : 1 yrs | Health: 88%
  Traits : Playful
---------------------------------------
Press any key to return...
```

Figure 3: *"Card Style" pet profiles showing vertical alignment of pet attributes.*

4. Source Code

The complete C++ source code for the Paw-Some Pets & Co. application is listed below.

```
#include <iostream>
#include <conio.h>
using namespace std;
int main() // main
{
    // DATA-Structures
    string mainoptions;
    int max_capacity = 1000;

    string petNames[max_capacity] = {"Whiskers", "Tracker", "Duke", "Luna"};
    string petTypes[max_capacity] = {"Domestic", "Field & Sport", "Field & Sport", "Domestic"};
    string petBreeds[max_capacity] = {"Persian", "Pointer", "Labrador", "Siamese"};
    int petAges[max_capacity] = {2, 3, 2, 1};
    float petHealth[max_capacity] = {92.5, 98.0, 99.0, 88.0};
    string petTraits[max_capacity] = {"Calm", "Fast Tracker", "Great Swimmer", "Playful"};
    double donatedMoney = 100;
    double totalAdoptedMoney = 500;
    double fixFeeForAdoption = 1000;
    // Transaction History Data Structures
    int history_size = 10;
    string historyType[history_size]; // "Adoption" or "Donation"
    double historyAmount[history_size];
    string historyDetail[history_size]; // Pet name or Donor note
    int historyCount = 0;               // Tracks how many transactions have occurred

    int petCount = 4;

    while (true) // While loop for main homescreen
    {
        system("cls");
        cout << "\n=====================================================" << endl;
        cout << "               PAW-SOME PETS & CO.                   " << endl;
        cout << "        - Pet Management & Adoption System -         " << endl;
        cout << "=====================================================" << endl;
        cout << "\n  [ LOGIN AS ]" << endl;
        cout << "  1. Administrative Staff" << endl;
        cout << "  2. Pet Donor" << endl;
        cout << "  3. Customer / Adopter" << endl;
        cout << "  4. Exit Application" << endl;
        cout << "\n-----------------------------------------------------" << endl;
        cout << "  [?] Select your role --> ";
        cin >> mainoptions;

        // MENU-Option decider using conditions
        if (mainoptions == "1") // filtering the admin menu using condition
        {
            bool loginSuccess = false;

            for (int i = 0; i < 3; i++) // loop for limiting credentials
            {
                system("cls");
                string adminPass;
                cout << "\n=====================================================" << endl;
                cout << "               ADMINISTRATIVE ACCESS                 " << endl;
```

```
                cout << "=====================================================" << endl;
                cout << "\n  [!] Security Check Required" << endl;
                cout << "Admin Menu : (LOGIN ATTEMPT : " << i + 1 << ")" << endl;
                cout << "Enter User name: ";
                string username;
                cin >> username;
                cout << "Enter Password: ";
                string password;
                cin >> password;

                if (username == "admin" && password == "123") // comparing if credentials are
correct
                {
                    cout << "Successfully Logged in!" << endl;
                    system("cls");
                    loginSuccess = true;
                    break;
                }

                else
                {
                    if (i < 2)
                    {
                        cout << "You've Entered wrong username or password!" << endl;
                        cout << "\nPress any key to continue...";
                        getch();
                    }
                }
            }

            if (loginSuccess)
            {
                while (true)
                {
                    system("cls");
                    // admincode
                    string adminoptions;
                    cout << "\n=====================================================" << endl;
                    cout << "                ADMINISTRATIVE DASHBOARD            " << endl;
                    cout << "=====================================================" << endl;
                    cout << "  [1] Display All Pets" << endl;
                    cout << "  [2] Search Pet by Name" << endl;
                    cout << "  [3] Sorting Pets (Age/Health)" << endl;
                    cout << "  [4] Update Pet Information" << endl;
                    cout << "  [5] Delete a Pet Record" << endl;
                    cout << "  [6] Inventory Statistics" << endl;
                    cout << "  [7] View Donation Bank" << endl;
                    cout << "  [8] View Adoption Revenue" << endl;
                    cout << "  [9] View Transaction History" << endl;
                    cout << "  [10] Return to Main Menu" << endl;
                    cout << "-----------------------------------------------------" << endl;
                    cout << "  [?] Select an action --> ";
                    cin >> adminoptions;

                    if (adminoptions == "1") // displaying all pet data in vertical box order
                    {
                        system("cls");
                        cout << "================================================" << endl;
```

```
                        cout << "           CURRENT PET INVENTORY                " << endl;
                        cout << "================================================" << endl;

                        cout << "\nTotal Pets: " << petCount << endl;
                        for (int i = 0; i < petCount; i++)
                        {

                            cout << "[ ID: " << i + 1 << " ]-------------------------------------
---" << endl;
                            cout << "  Name   : " << petNames[i] << endl;
                            cout << "  Type   : " << petTypes[i] << " (" << petBreeds[i] << ")"
<< endl;
                            cout << "  Status : " << petAges[i] << " yrs | Health: " <<
petHealth[i] << "%" << endl;
                            cout << "  Traits : " << petTraits[i] << endl;
                        }
                        cout << "----------------------------------------" << endl;
                        cout << "Press any key to return...";
                        getch();
                        system("cls");
                    }
                    else if (adminoptions == "2") // search by name
                    {
                        while (true)
                        {
                            system("cls");
                            int searchmenuoption;
                            cout << "=====================================\n";
                            cout << "          Searching Pets             \n";
                            cout << "=====================================\n\n";
                            cout << "---> 1.Search\n";
                            cout << "---> 2.Go back\n";
                            cout << "  Enter you choice : ";
                            cin >> searchmenuoption;
                            if (searchmenuoption == 1)
                            {
                                system("cls");

                                cout << "Enter the name you want to search: ";
                                string searchName;
                                cin.ignore();
                                getline(cin, searchName);

                                bool isFound = false;
                                int found_index = -1;
                                for (int i = 0; i < petCount; i++)
                                {
                                    string currentPet = petNames[i];

                                    // MANUAL ASCII solution to solve the problem of searching
with case sensitivness
                                    //  Converting BOTH strings to lowercase manually for
comparison
                                    string searchLower = searchName;
                                    string storedLower = currentPet;

                                    for (int j = 0; j < searchLower.length(); j++)
                                    {
```

```
                                    if (searchLower[j] >= 65 && searchLower[j] <= 90) // If
it's Capital (A-Z)
                                        searchLower[j] = searchLower[j] + 32;         //
Making it lowercase
                                }
                                for (int j = 0; j < storedLower.length(); j++)
                                {
                                    if (storedLower[j] >= 65 && storedLower[j] <= 90) // If
it's Capital (A-Z)
                                        storedLower[j] = storedLower[j] + 32;         //
Making it lowercase
                                }
                                if (storedLower == searchLower)
                                {
                                    cout << "[ ID: " << i + 1 << " ]------------------------
-------" << endl;
                                    cout << "  Name   : " << petNames[i] << endl;
                                    cout << "  Type   : " << petTypes[i] << " (" <<
petBreeds[i] << ")" << endl;
                                    cout << "  Status : " << petAges[i] << " yrs | Health: "
<< petHealth[i] << "%" << endl;
                                    cout << "  Traits : " << petTraits[i] << endl;
                                    isFound = true;
                                    found_index = i;
                                    cout << "\nPress any key to go back...";
                                    getch();
                                    break;
                                }
                            }
                            if (isFound == false)
                            {
                                cout << "No record Found for : " << searchName << endl;
                                cout << "Press any key to go back...";
                                getch();
                            }
                        }
                        else if (searchmenuoption == 2)
                        {
                            break;
                        }
                        else
                        {
                            cout << "Wrong selection!\n";
                            cout << "Press any key to continue...";
                            getch();
                        }
                    }
                }
                else if (adminoptions == "3") // sort pets by health or age
                {
                    // bubble sorting out the pets
                    while (true)
                    {
                        system("cls");
                        int sortOptions;
                        cout << "========================================\n";
                        cout << "           Sorting of Pets              \n";
                        cout << "========================================\n";
```

```
                                cout << "---> 1.Sort by Health?(highest first) \n";
                                cout << "---> 2.Sort by Age?\n";
                                cout << "---> 3.Go Back\n";
                                cout << "\nEnter you selection : ";
                                cin >> sortOptions;
                                if (sortOptions == 1)
                                { // bubble sorting pets by health
                                    for (int i = 0; i < petCount - 1; i++)
                                    {
                                        for (int j = 0; j < petCount - i - 1; j++)
                                        {
                                            if (petHealth[j] < petHealth[j + 1])
                                            {
                                                // 1. Swap Health
                                                float tempHealth = petHealth[j];
                                                petHealth[j] = petHealth[j + 1];
                                                petHealth[j + 1] = tempHealth;

                                                // 2. Swap Names || swapping all aswell to maintain
the data in parrallel arrays
                                                string tempName = petNames[j];
                                                petNames[j] = petNames[j + 1];
                                                petNames[j + 1] = tempName;

                                                // 3. Swap Types
                                                string tempType = petTypes[j];
                                                petTypes[j] = petTypes[j + 1];
                                                petTypes[j + 1] = tempType;

                                                // 4. Swap Breeds
                                                string tempBreed = petBreeds[j];
                                                petBreeds[j] = petBreeds[j + 1];
                                                petBreeds[j + 1] = tempBreed;

                                                // 5. Swap Ages
                                                int tempAge = petAges[j];
                                                petAges[j] = petAges[j + 1];
                                                petAges[j + 1] = tempAge;

                                                // 6. Swap Traits
                                                string tempTrait = petTraits[j];
                                                petTraits[j] = petTraits[j + 1];
                                                petTraits[j + 1] = tempTrait;
                                            }
                                        }
                                    }
                                    cout << "Successfully sorted All by health! Use Display All to
check..." << endl;
                                    cout << "\nPress any key to continue...";
                                    getch();
                                }
                                else if (sortOptions == 2) // bubble sorting pets by age
                                {
                                    int sortageOptions;
                                    system("cls");
                                    // menu for age sorting youngest to oldest or oldest to youngest
                                    cout << "---> 1. Oldest to Youngest\n";
                                    cout << "---> 2. Youngest to Oldest\n";
```

```
                                cout << "       Enter your selection : ";
                                cin >> sortageOptions;
                                if (sortageOptions == 1)
                                {
                                    for (int i = 0; i < petCount - 1; i++)
                                    {
                                        for (int j = 0; j < petCount - i - 1; j++)
                                        {
                                            if (petAges[j] < petAges[j + 1])
                                            {
                                                // 5. Swap Ages
                                                int tempAge = petAges[j];
                                                petAges[j] = petAges[j + 1];
                                                petAges[j + 1] = tempAge;

                                                // 2. Swap Names || swapping all aswell to
maintain the data in parrallel arrays
                                                string tempName = petNames[j];
                                                petNames[j] = petNames[j + 1];
                                                petNames[j + 1] = tempName;

                                                // 3. Swap Types
                                                string tempType = petTypes[j];
                                                petTypes[j] = petTypes[j + 1];
                                                petTypes[j + 1] = tempType;

                                                // 4. Swap Breeds
                                                string tempBreed = petBreeds[j];
                                                petBreeds[j] = petBreeds[j + 1];
                                                petBreeds[j + 1] = tempBreed;

                                                // 1. Swap Health
                                                float tempHealth = petHealth[j];
                                                petHealth[j] = petHealth[j + 1];
                                                petHealth[j + 1] = tempHealth;

                                                // 6. Swap Traits
                                                string tempTrait = petTraits[j];
                                                petTraits[j] = petTraits[j + 1];
                                                petTraits[j + 1] = tempTrait;
                                            }
                                        }
                                    }
                                    cout << "Successfully sorted All by age(Oldest to Youngest)
!! \n Use Display All to check..." << endl;
                                    cout << "\nPress any key to continue...";
                                    getch();
                                }
                                else if (sortageOptions == 2)
                                {
                                    for (int i = 0; i < petCount - 1; i++)
                                    {
                                        for (int j = 0; j < petCount - i - 1; j++)
                                        {
                                            if (petAges[j] > petAges[j + 1])
                                            {
                                                // 5. Swap Ages
                                                int tempAge = petAges[j];
```

```
                                                petAges[j] = petAges[j + 1];
                                                petAges[j + 1] = tempAge;

                                                // 2. Swap Names || swapping all aswell to
maintain the data in parrallel arrays
                                                string tempName = petNames[j];
                                                petNames[j] = petNames[j + 1];
                                                petNames[j + 1] = tempName;

                                                // 3. Swap Types
                                                string tempType = petTypes[j];
                                                petTypes[j] = petTypes[j + 1];
                                                petTypes[j + 1] = tempType;

                                                // 4. Swap Breeds
                                                string tempBreed = petBreeds[j];
                                                petBreeds[j] = petBreeds[j + 1];
                                                petBreeds[j + 1] = tempBreed;

                                                // 1. Swap Health
                                                float tempHealth = petHealth[j];
                                                petHealth[j] = petHealth[j + 1];
                                                petHealth[j + 1] = tempHealth;

                                                // 6. Swap Traits
                                                string tempTrait = petTraits[j];
                                                petTraits[j] = petTraits[j + 1];
                                                petTraits[j + 1] = tempTrait;
                                            }
                                        }
                                    }
                                    cout << "Successfully sorted All by age(Youngest to oldest)
!! \n Use Display All to check..." << endl;
                                    cout << "\nPress any key to continue...";
                                    getch();
                                }
                            }
                            else if (sortOptions == 3)
                            {
                                break;
                            }
                            else
                            {
                                cout << "Wrong Selection! Enter again! \n";
                                cout << "Press any key to continue...";
                                getch();
                            }
                        }
                    }
                    else if (adminoptions == "4") // update pet information
                    {
                        system("cls");
                        cout << "=====================================\n";
                        cout << "           Updating Pets             \n";
                        cout << "=====================================\n\n";
                        cout << "Enter the name you want to update: ";
                        string updateName;
                        cin.ignore();
```

```
                        getline(cin, updateName);
                        bool isFound = false;
                        int found_index = -1;
                        for (int i = 0; i < petCount; i++)
                        {
                            string currentPet = petNames[i];

                            // MANUAL ASCII solution to solve the problem of searching name for
updating with case sensitivness
                            //  Converting BOTH strings to lowercase manually for comparison
                            string searchLower = updateName;
                            string storedLower = currentPet;

                            for (int j = 0; j < searchLower.length(); j++)
                            {
                                if (searchLower[j] >= 65 && searchLower[j] <= 90) // If it's
Capital (A-Z)
                                    searchLower[j] = searchLower[j] + 32;         // Making it
lowercase
                            }
                            for (int j = 0; j < storedLower.length(); j++)
                            {
                                if (storedLower[j] >= 65 && storedLower[j] <= 90) // If it's
Capital (A-Z)
                                    storedLower[j] = storedLower[j] + 32;         // Making it
lowercase
                            }
                            if (storedLower == searchLower)
                            {
                                cout << "OLD Record of " << updateName; // displaying old record
first to confirm to the user
                                cout << "[ ID: " << i + 1 << " ]\n-------------------------------
-" << endl;
                                cout << "  Name   : " << petNames[i] << endl;
                                cout << "  Type   : " << petTypes[i] << " (" << petBreeds[i] <<
")" << endl;
                                cout << "  Status : " << petAges[i] << " yrs | Health: " <<
petHealth[i] << "%" << endl;
                                cout << "  Traits : " << petTraits[i] << endl;
                                isFound = true;
                                found_index = i;
                                cout << "\n Press any key to continue...";
                                getch();
                                system("cls");
                                cout << "=====================================\n";
                                cout << "           Updating Pets             \n";
                                cout << "=====================================\n\n";
                                cout << "Enter the new name for updating : "; // getting new data
from the user
                                string tempPetname;
                                getline(cin, tempPetname);
                                cout << "Enter the new type for updating : ";
                                string tempPettype;
                                getline(cin, tempPettype);
                                cout << "Enter the new pet breed for updating : ";
                                string tempPetbreeds;
                                getline(cin, tempPetbreeds);
                                cout << "Enter the new pet age for updating : ";
```

```
                                int tempPetage;
                                cin >> tempPetage;
                                cin.ignore();
                                cout << "Enter the new pet health for updating : ";
                                double tempPethealth;
                                cin >> tempPethealth;
                                cin.ignore();
                                cout << "Enter the new trait of the pet for updating : ";
                                string tempPettrait;
                                getline(cin, tempPettrait);
                                petNames[i] = tempPetname; // storing new data from temp vars to
original arrays
                                petTypes[i] = tempPettype;
                                petBreeds[i] = tempPetbreeds;
                                petAges[i] = tempPetage;
                                petHealth[i] = tempPethealth;
                                petTraits[i] = tempPettrait;
                                cout << "Successfully updated the Data...  Check 'search by name'
or 'display all' to confirm";

                                break;
                            }
                        }
                        if (isFound == false)
                        {
                            cout << "No record Found for : " << updateName << endl;
                        }
                        cout << "\nPress any key to return to Admin Menu...";
                        getch();
                        system("cls");
                    }
                    else if (adminoptions == "5") // delete pet info
                    {
                        system("cls");
                        // code for deleting pet info
                        cout << "=====================================\n";
                        cout << "           Removing Pets             \n";
                        cout << "=====================================\n\n";
                        cout << "Enter the pet name you want to delete: ";
                        string deleteName;
                        cin.ignore();
                        getline(cin, deleteName);
                        bool isFound = false;
                        int found_index = -1;
                        for (int i = 0; i < petCount; i++)
                        {
                            string currentPet = petNames[i];

                            // MANUAL ASCII solution to solve the problem of searching with case
sensitivness
                            //  Converting BOTH strings to lowercase manually for comparison
                            string searchLower = deleteName;
                            string storedLower = currentPet;

                            for (int j = 0; j < searchLower.length(); j++)
                            {
                                if (searchLower[j] >= 65 && searchLower[j] <= 90) // If it's
Capital (A-Z)
```

```
                                searchLower[j] = searchLower[j] + 32;         // Making it
lowercase
                            }
                            for (int j = 0; j < storedLower.length(); j++)
                            {
                                if (storedLower[j] >= 65 && storedLower[j] <= 90) // If it's
Capital (A-Z)
                                    storedLower[j] = storedLower[j] + 32;         // Making it
lowercase
                            }
                            if (storedLower == searchLower)
                            {
                                cout << "        Record of " << deleteName << endl;
                                cout << "[ ID: " << i + 1 << " ]\n-------------------------------
-" << endl;
                                cout << "  Name   : " << petNames[i] << endl;
                                cout << "  Type   : " << petTypes[i] << " (" << petBreeds[i] <<
")" << endl;
                                cout << "  Status : " << petAges[i] << " yrs | Health: " <<
petHealth[i] << "%" << endl;
                                cout << "  Traits : " << petTraits[i] << endl;
                                isFound = true;
                                found_index = i;
                                cout << "press any key to continue...";
                                getch();
                                system("cls");
                                for (int i = found_index; i < petCount - 1; i++)
                                {
                                    petNames[i] = petNames[i + 1];
                                    petTypes[i] = petTypes[i + 1];
                                    petBreeds[i] = petBreeds[i + 1];
                                    petAges[i] = petAges[i + 1];
                                    petHealth[i] = petHealth[i + 1];
                                    petTraits[i] = petTraits[i + 1];
                                }
                                petCount--;
                                cout << "=====================================\n";
                                cout << "           Removing Pets             \n";
                                cout << "=====================================\n\n";
                                cout << "Succesfully Deleted Record of " << deleteName;
                                break;
                            }
                        }
                        if (isFound == false)
                        {
                            cout << "No record Found for : " << deleteName << endl;
                        }
                        cout << "\nPress any key to return to Admin Menu...";
                        getch();
                        system("cls");
                    }
                    else if (adminoptions == "6") // inventory statistics
                    {
                        system("cls");
                        // code for inventory statistics
                        while (true)
                        {
                            int domesticCount = 0;
```

```
                            int fieldsportCount = 0;
                            int oldidx = 0;
                            int youngidx = 0;
                            if (petCount == 0)
                            {
                                cout << "Inventory is empty!";
                                break;
                            }
                            else
                            {
                                for (int i = 0; i < petCount; i++) // finding domestic & Field
and sport pets count
                                {
                                    if (petTypes[i] == "Domestic" || petTypes[i] == "domestic")
                                    {
                                        domesticCount++;
                                    }
                                    if (petTypes[i] == "Field & Sport" || petTypes[i] == "Field
and Sport")

                                    {
                                        fieldsportCount++;
                                    }
                                }
                                for (int i = 0; i < petCount; i++) // finding oldest and youngest
pets
                                {
                                    if (petAges[i] > petAges[oldidx])
                                    {
                                        oldidx = i;
                                    }
                                    if (petAges[i] < petAges[youngidx])
                                    {
                                        youngidx = i;
                                    }
                                }

                                // displaying the inventory statistics
                                cout << "=====================================\n";
                                cout << "          Inventory Statistics         \n";
                                cout << "=====================================\n\n";
                                cout << " --->   Domestic pets : " << domesticCount << "\n";
                                cout << " --->   Field & Sport pets : " << fieldsportCount <<
"\n";
                                cout << " --->   Oldest pet : " << petNames[oldidx] << "\n";
                                cout << " --->   Youngest pet : " << petNames[youngidx] <<
"\n\n";
                                cout << "=====================================\n\n";
                                cout << " Press any key to go back..." << endl;
                                getch();
                                break;
                            }
                        }
                    }
                    else if (adminoptions == "7") // showing donation money
                    {
                        system("cls");
                        // code for showing donated money
```

```
                    cout << "=============================================\n";
                    cout << "               Donation Money                \n";
                    cout << "=============================================\n";
                    cout << "   Donation Bank Balance : " << donatedMoney << " PKR";
                    cout << "\nPress any key to go back...";
                    getch();
                }
                else if (adminoptions == "8") // adoption money total
                {
                    system("cls");
                    // code for showing adoption money
                    cout << "=============================================\n";
                    cout << "               Adoption Fee Money                \n";
                    cout << "=============================================\n";
                    cout << "   Adoption Fee Bank Balance : " << totalAdoptedMoney << " PKR";
                    cout << "\nPress any key to go back...";
                    getch();
                }
                else if (adminoptions == "9") // View Transaction History
                {
                    system("cls");
                    cout << "================================================" << endl;
                    cout << "           RECENT TRANSACTION HISTORY           " << endl;
                    cout << "================================================" << endl;

                    if (historyCount == 0)
                    {
                        cout << "\n   No transactions recorded yet." << endl;
                    }
                    else
                    {
                        for (int i = 0; i < historyCount; i++)
                        {
                            cout << i + 1 << ". [" << historyType[i] << "]" << endl;
                            cout << "    Amount : " << historyAmount[i] << " PKR" << endl;
                            cout << "    Details: " << historyDetail[i] << endl;
                            cout << "----------------------------------------------------" <<
endl;
                        }
                    }
                    cout << "\nPress any key to return...";
                    getch();
                }
                else if (adminoptions == "10") // break to main menu
                {
                    break;
                }
                else
                {
                    cout << "Wrong selection! Enter again..." << endl;
                    cout << "Press any key to continue...";
                    getch();
                }
            }
        }
        else if (loginSuccess != true)
        {
```

```
        system("cls");
        cout << "Maximum login attempts reached. Access Denied!" << endl;
        cout << "Press any key to return to the main menu...";
        getch();
    }
}
else if (mainoptions == "2") // filtering the donor menu using condition
{
    // donorcode
    while (true)
    {
        system("cls");
        int donorOptions;
        cout << "\n=====================================================" << endl;
        cout << "                 PET DONOR DASHBOARD                 " << endl;
        cout << "=====================================================" << endl;
        cout << "  [1] Donate a Pet" << endl;
        cout << "  [2] Donate Money" << endl;
        cout << "  [3] View Inventory Capacity" << endl;
        cout << "  [4] Return to Main Menu" << endl;
        cout << "-----------------------------------------------------" << endl;
        cout << "  [?] Select an action --> ";
        cin >> donorOptions;
        if (donorOptions == 1)
        {
            if (petCount < max_capacity)
            {
                cin.ignore();
                cout << "Enter the name of the pet you want to Donate : ";
                string donateName;
                getline(cin, donateName);
                cout << "Enter the Pet's type : ";
                string donateType;
                getline(cin, donateType);
                cout << "Enter the Pet's breed : ";
                string donateBreed;
                getline(cin, donateBreed);
                cout << "Enter the Pet's age : ";
                int donateAge;
                cin >> donateAge;
                cout << "Enter the Pet's Health(in %) : ";
                double donateHealth;
                cin >> donateHealth;
                cin.ignore();
                cout << "Enter the Pet's trait (Usual Behaviour) : ";
                string donateTrait;
                getline(cin, donateTrait);
                petNames[petCount] = donateName;
                petTypes[petCount] = donateType;
                petBreeds[petCount] = donateBreed;
                petAges[petCount] = donateAge;
                petHealth[petCount] = donateHealth;
                petTraits[petCount] = donateTrait;
                petCount++;
                cout << "\n\n Successfully Donted Pet For a Good Cause <3";
            }
            else
            {
```

```
                        cout << "==============================================" << endl;
                        cout << "            NOTICE: SHELTER FULL               " << endl;
                        cout << "==============================================" << endl;
                        cout << "We are currently at our maximum capacity of " << max_capacity <<
" pets." << endl;
                        cout << "We cannot accept new donations at this time." << endl;
                        cout << "\n[!] ALERT: A request has been sent to the Manager" << endl;
                        cout << "    to increase storage capacity soon." << endl;
                        cout << "\nPress any key to return...";
                        getch();
                    }
                }
                else if (donorOptions == 2)
                {
                    system("cls");
                    string donateMoneyselection;
                    cout << "=====================================\n";
                    cout << "           Pet Donor Menu             \n";
                    cout << "=====================================\n\n";
                    double tempDonatedMoney;
                    cout << "\n Enter amount of money to donate(in PKR) : ";
                    cin >> tempDonatedMoney;
                    donatedMoney = donatedMoney + tempDonatedMoney;
                    if (historyCount < history_size)
                    {
                        historyType[historyCount] = "Donation";
                        historyAmount[historyCount] = tempDonatedMoney;
                        historyDetail[historyCount] = "General Support";
                        historyCount++;
                    }
                    cout << "\nSuccessfully Donated " << tempDonatedMoney << " PKR, For a Good
Cause <3\n =====================================\n     May Allah Give You More!!
\n=====================================\n";
                    cout << "Press any key to continue...";
                    getch();
                }
                else if (donorOptions == 3)
                {
                    int tempRemainingCapacity;
                    if (max_capacity - petCount > 0)
                    {
                        tempRemainingCapacity = max_capacity - petCount;
                    }
                    else
                    {
                        tempRemainingCapacity = 0;
                    }
                    cout << "=============================================\n";
                    cout << "          Remaining Shelter Capacity          \n";
                    cout << "=============================================\n";
                    cout << "---> Remaining Capacity : " << tempRemainingCapacity;
                    cout << "\nPress any key to continue...";
                    getch();
                }
                else if (donorOptions == 4)
                {
                    break;
                }
```

```
        else
        {
            cout << "Wrong Choice !! Please enter again...";
            cout << "\n Press any key to continue...";
            getch();
        }
    }
}
else if (mainoptions == "3") // filtering the customer menu using condition
{
    // customercode
    while (true)
    {
        system("cls");
        int customerOptions;
        cout << "\n=====================================================" << endl;
        cout << "              WELCOME TO PAW-SOME                   " << endl;
        cout << "           Find Your New Best Friend!               " << endl;
        cout << "=====================================================" << endl;
        cout << "  [1] View All Pets" << endl;
        cout << "  [2] Search For a Specific Pet" << endl;
        cout << "  [3] Adopt a Pet" << endl;
        cout << "  [4] Donate Money for the Shelter" << endl;
        cout << "  [5] Return to Main Menu" << endl;
        cout << "-----------------------------------------------------" << endl;
        cout << "  [?] Select an action --> ";
        cin >> customerOptions;

        if (customerOptions == 1)
        {
            system("cls");
            cout << "\n=====================================================" << endl;
            cout << "                CURRENT PET INVENTORY               " << endl;
            cout << "=====================================================" << endl;

            for (int i = 0; i < petCount; i++)
            {
                cout << "  [" << i + 1 << "] Name   : " << petNames[i] << endl;
                cout << "      Type   : " << petTypes[i] << endl;
                cout << "      Breed  : " << petBreeds[i] << endl;
                cout << "      Age    : " << petAges[i] << " years" << endl;
                cout << "      Health : " << petHealth[i] << "%" << endl;
                cout << "      Trait  : " << petTraits[i] << endl;
                cout << "-----------------------------------------------------" << endl;
            }
            cout << "\n  Press any key to return to Dashboard...";
            getch();
        }
        else if (customerOptions == 2) // searching for a specific pet
        {
            while (true)
            {
                system("cls");
                int searchmenuoption;
                cout << "=====================================\n";
                cout << "          Searching Pets             \n";
                cout << "=====================================\n\n";
                cout << "---> 1.Search\n";
```

```
                        cout << "---> 2.Go back\n";
                        cout << "  Enter you choice : ";
                        cin >> searchmenuoption;
                        if (searchmenuoption == 1)
                        {
                            system("cls");

                            cout << "Enter the name you want to search: ";
                            string searchName;
                            cin.ignore();
                            getline(cin, searchName);

                            bool isFound = false;
                            int found_index = -1;
                            for (int i = 0; i < petCount; i++)
                            {
                                string currentPet = petNames[i];

                                // MANUAL ASCII solution to solve the problem of searching with
case sensitivness
                                //  Converting BOTH strings to lowercase manually for comparison
                                string searchLower = searchName;
                                string storedLower = currentPet;

                                for (int j = 0; j < searchLower.length(); j++)
                                {
                                    if (searchLower[j] >= 65 && searchLower[j] <= 90) // If it's
Capital (A-Z)
                                        searchLower[j] = searchLower[j] + 32;         // Making
it lowercase
                                }
                                for (int j = 0; j < storedLower.length(); j++)
                                {
                                    if (storedLower[j] >= 65 && storedLower[j] <= 90) // If it's
Capital (A-Z)
                                        storedLower[j] = storedLower[j] + 32;         // Making
it lowercase
                                }
                                if (storedLower == searchLower)
                                {
                                    cout << "[ ID: " << i + 1 << " ]-----------------------------
---" << endl;
                                    cout << "  Name   : " << petNames[i] << endl;
                                    cout << "  Type   : " << petTypes[i] << " (" << petBreeds[i]
<< ")" << endl;
                                    cout << "  Status : " << petAges[i] << " yrs | Health: " <<
petHealth[i] << "%" << endl;
                                    cout << "  Traits : " << petTraits[i] << endl;
                                    isFound = true;
                                    found_index = i;
                                    cout << "\nPress any key to go back...";
                                    getch();
                                    break;
                                }
                            }
                            if (isFound == false)
                            {
                                cout << "No record Found for : " << searchName << endl;
```

```
                    cout << "Press any key to go back...";
                    getch();
                }
            }
            else if (searchmenuoption == 2)
            {
                break;
            }
            else
            {
                cout << "Wrong selection!\n";
                cout << "Press any key to continue...";
                getch();
            }
        }
    }
    else if (customerOptions == 3) // adopting a pet
    {
        system("cls");
        // code for adopting pet
        cout << "=====================================\n";
        cout << "           Adopting Pets             \n";
        cout << "=====================================\n\n";
        cout << "Enter the pet name you want to adopt: ";
        string adoptName;
        cin.ignore();
        getline(cin, adoptName);
        bool isFound = false;
        int found_index = -1;
        for (int i = 0; i < petCount; i++)
        {
            string currentPet = petNames[i];

            // MANUAL ASCII solution to solve the problem of searching with case
sensitivness
            //  Converting BOTH strings to lowercase manually for comparison
            string searchLower = adoptName;
            string storedLower = currentPet;

            for (int j = 0; j < searchLower.length(); j++)
            {
                if (searchLower[j] >= 65 && searchLower[j] <= 90) // If it's Capital
(A-Z)
                    searchLower[j] = searchLower[j] + 32;         // Making it
lowercase
            }
            for (int j = 0; j < storedLower.length(); j++)
            {
                if (storedLower[j] >= 65 && storedLower[j] <= 90) // If it's Capital
(A-Z)
                    storedLower[j] = storedLower[j] + 32;         // Making it
lowercase
            }
            if (storedLower == searchLower)
            {
                string confirmationMsg;
                cout << "    FIXED Fee for Adoption : " << fixFeeForAdoption << "\n";
                cout << "        Record of " << adoptName << endl;
```

```
                            cout << "[ ID: " << i + 1 << " ]\n--------------------------------"
<< endl;
                            cout << "  Name   : " << petNames[i] << endl;
                            cout << "  Type   : " << petTypes[i] << " (" << petBreeds[i] << ")"
<< endl;
                            cout << "  Status : " << petAges[i] << " yrs | Health: " <<
petHealth[i] << "%" << endl;
                            cout << "  Traits : " << petTraits[i] << endl;
                            isFound = true;
                            found_index = i;
                            cout << "press any key to continue...";
                            getch();
                            system("cls");
                            cout << "Confirm Payment Of " << fixFeeForAdoption << " for Adopting
" << petNames[found_index] << " ?(yes/no) : ";
                            getline(cin, confirmationMsg);
                            if (confirmationMsg == "yes")
                            {
                                totalAdoptedMoney = totalAdoptedMoney + fixFeeForAdoption;
                                if (historyCount < history_size)
                                {
                                    historyType[historyCount] = "Adoption";
                                    historyAmount[historyCount] = fixFeeForAdoption;
                                    historyDetail[historyCount] = "Adopted: " + adoptName;
                                    historyCount++;
                                }
                                cout << "====================================\n";
                                cout << "           Adopting Pets             \n";
                                cout << "====================================\n\n";

                                cout << "\n  Congratulations on Adopting " << adoptName << " It's
" << petTraits[found_index] << " \n We are sure it will be Frank with you right after!!";
                                for (int i = found_index; i < petCount - 1; i++)
                                {
                                    petNames[i] = petNames[i + 1];
                                    petTypes[i] = petTypes[i + 1];
                                    petBreeds[i] = petBreeds[i + 1];
                                    petAges[i] = petAges[i + 1];
                                    petHealth[i] = petHealth[i + 1];
                                    petTraits[i] = petTraits[i + 1];
                                }
                                petCount--;

                                break;
                            }
                            else
                            {

                                break;
                            }
                        }
                    }
                    if (isFound == false)
                    {
                        cout << "No record Found for : " << adoptName << endl;
                    }
                    cout << "\nPress any key to return to Main Menu...";
                    getch();
```

```
                    system("cls");
                }
                else if (customerOptions == 4)
                {
                    system("cls");
                    string donateMoneyselection;
                    cout << "=====================================\n";
                    cout << "           Donation Menu             \n";
                    cout << "=====================================\n\n";
                    double tempDonatedMoney;
                    cout << "\n Enter amount of money to donate(in PKR) : ";
                    cin >> tempDonatedMoney;
                    donatedMoney = donatedMoney + tempDonatedMoney;
                    if (historyCount < history_size)
                    {
                        historyType[historyCount] = "Donation";
                        historyAmount[historyCount] = tempDonatedMoney;
                        historyDetail[historyCount] = "Customer General Support";
                        historyCount++;
                    }
                    cout << "\nSuccessfully Donated " << tempDonatedMoney << " PKR, For a Good
Cause <3\n =====================================\n     May Allah Give You More!!
\n=====================================\n";
                    cout << "Press any key to continue...";
                    getch();
                }
                else if (customerOptions == 5)
                {
                    break;
                }
                else
                {
                    cout << "Wrong choice!! Please enter again\n";
                    cout << "Press any key to continue...";
                    getch();
                }
            }
        }
        // loop breaker i.e the exit option
        else if (mainoptions == "4") // filtering the exit option using condition and logging out
        {
            break;
        }
        else // else condition if choice entered was wrong
        {
            cout << "Wrong Choice... Please enter again!!" << endl
                 << "Press Any Key to Continue...";
            getch();
        }
    }
    cout << "THANKS for using Paw-Some APPLICATION!!" << endl
         << endl
         << "Press any key to Exit...";
    getch();
    // screen clearing after exiting the loop
    system("cls");
}
```